

PPL LAB 5 230905290

Rohan Shenoy

230905290

CSE D1

Roll no 32

Q1]

1. Write a program in CUDA to add two vectors of length N using
a) block size as N b) N threads

CODE]

```
#include <stdio.h>
#include <cuda_runtime.h>

__global__ void vectorAdd(int *a, int *b, int *c, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        c[i] = a[i] + b[i];
    }
}

int main() {
    int n;
    printf("Enter vector length N: ");
    scanf("%d", &n);

    size_t size = n * sizeof(int);
    int *h_a = (int*)malloc(size), *h_b = (int*)malloc(size), *h_c = (int*)malloc(size);

    for (int i = 0; i < n; i++)
    {
        h_a[i] = i;
        h_b[i] = i * 2;
    }

    int *d_a, *d_b, *d_c;
    cudaMalloc((void**)&d_a, size);
    cudaMalloc((void**)&d_b, size);
    cudaMalloc((void**)&d_c, size);

    cudaMemcpy(d_a, h_a, size, cudaMemcpyHostToDevice);
```

```

cudaMemcpy(d_b, h_b, size, cudaMemcpyHostToDevice);

printf("\n--- Output for Case A (1 Block, %d Threads) ---\n", n);
vectorAdd<<<1, n>>>(d_a, d_b, d_c, n);
cudaMemcpy(h_c, d_c, size, cudaMemcpyDeviceToHost);
for (int i = 0; i < n; i++) printf("%d ", h_c[i]);

printf("\n\n--- Output for Case B (%d Blocks, 1 Thread each) ---\n", n);
vectorAdd<<<n, 1>>>(d_a, d_b, d_c, n);
cudaMemcpy(h_c, d_c, size, cudaMemcpyDeviceToHost);
for (int i = 0; i < n; i++) printf("%d ", h_c[i]);
printf("\n");

free(h_a); free(h_b); free(h_c);
cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
return 0;
}

```

OUTPUT]

```

6CSED1@debian:~/Desktop/ppl_lab/lab5$ nvcc q1.cu
6CSED1@debian:~/Desktop/ppl_lab/lab5$ ./a.out
Enter vector length N: 5

--- Output for Case A (1 Block, 5 Threads) ---
0 3 6 9 12

--- Output for Case B (5 Blocks, 1 Thread each) ---
0 3 6 9 12
6CSED1@debian:~/Desktop/ppl_lab/lab5$

```

Q2]

2. Implement a CUDA program to add two vectors of length N by keeping the number of threads per block as 256 (constant) and vary the number of blocks to handle N elements.

CODE]

```

#include <stdio.h>
#include <cuda_runtime.h>

__global__ void vectorAdd(int *a, int *b, int *c, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;

```

```

    if (i < n) {
        c[i] = a[i] + b[i];
    }
}

int main() {
    int n;
    printf("Enter vector length N: ");
    scanf("%d", &n);

    size_t size = n * sizeof(int);
    int *h_a = (int*)malloc(size), *h_b = (int*)malloc(size), *h_c = (int*)malloc(size);

    for (int i = 0; i < n; i++) {
        h_a[i] = i;
        h_b[i] = i * 2;
    }

    int *d_a, *d_b, *d_c;
    cudaMalloc((void**)&d_a, size);
    cudaMalloc((void**)&d_b, size);
    cudaMalloc((void**)&d_c, size);

    cudaMemcpy(d_a, h_a, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, h_b, size, cudaMemcpyHostToDevice);

    int threadsPerBlock = 256;
    int blocksPerGrid = (n + threadsPerBlock - 1) / threadsPerBlock;

    printf("Launching with %d blocks and %d threads per block\n", blocksPerGrid,
threadsPerBlock);

    vectorAdd<<<blocksPerGrid, threadsPerBlock>>>(d_a, d_b, d_c, n);

    cudaMemcpy(h_c, d_c, size, cudaMemcpyDeviceToHost);

    for (int i = 0; i < (n < 10 ? n : 10); i++) printf("%d ", h_c[i]);
    printf("\nDone.\n");

    free(h_a); free(h_b); free(h_c);
    cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
    return 0;
}

```

OUTPUT]

```
6CSED1@debian:~/Desktop/ppl_lab/lab5$ nvcc q2.cu
6CSED1@debian:~/Desktop/ppl_lab/lab5$ ./a.out
Enter vector length N: 3
Launching with 1 blocks and 256 threads per block
0 3 6
Done.
6CSED1@debian:~/Desktop/ppl_lab/lab5$
```

Q3]

3. Write a program in CUDA to process a 1D array containing angles in radians to generate sine of the angles in the output array. Use appropriate function.

CODE]

```
#include <stdio.h>
#include <cuda_runtime.h>
#include <math.h>

__global__ void computeSine(float *input, float *output, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        output[i] = sinf(input[i]);
    }
}

int main() {
    int n = 10;
    size_t size = n * sizeof(float);

    float *h_in = (float*)malloc(size);
    float *h_out = (float*)malloc(size);

    for (int i = 0; i < n; i++) {
        h_in[i] = i * (M_PI / (n - 1));
    }

    float *d_in, *d_out;
    cudaMalloc((void**)&d_in, size);
    cudaMalloc((void**)&d_out, size);

    cudaMemcpy(d_in, h_in, size, cudaMemcpyHostToDevice);
```

```

int threadsPerBlock = 256;
int blocksPerGrid = (n + threadsPerBlock - 1) / threadsPerBlock;

computeSine<<<blocksPerGrid, threadsPerBlock>>>(d_in, d_out, n);

cudaMemcpy(h_out, d_out, size, cudaMemcpyDeviceToHost);

printf("Radians \t Sine\n");
for (int i = 0; i < n; i++) {
    printf("%.4f \t %.4f\n", h_in[i], h_out[i]);
}

free(h_in); free(h_out);
cudaFree(d_in); cudaFree(d_out);

return 0;
}

```

OUTPUT]

```

6CSED1@debian:~/Desktop/ppl_lab/lab5$ nvcc q3.cu
6CSED1@debian:~/Desktop/ppl_lab/lab5$ ./a.out
Radians      Sine
0.0000      0.0000
0.3491      0.3420
0.6981      0.6428
1.0472      0.8660
1.3963      0.9848
1.7453      0.9848
2.0944      0.8660
2.4435      0.6428
2.7925      0.3420
3.1416      -0.0000
6CSED1@debian:~/Desktop/ppl_lab/lab5$

```

ADDITIONAL QUESTIONS

Q1]

1. Write a program in CUDA to perform linear algebra function of the form $y = \alpha x + y$, where x and y are vectors and α is a scalar value.

CODE]

```
#include <stdio.h>
#include <cuda_runtime.h>

__global__ void saxpy(int n, float a, float *x, float *y) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        y[i] = a * x[i] + y[i];
    }
}

int main() {
    int n = 1 << 20;
    float a = 2.5f;
    size_t size = n * sizeof(float);

    float *h_x = (float*)malloc(size);
    float *h_y = (float*)malloc(size);

    for (int i = 0; i < n; i++) {
        h_x[i] = 1.0f;
        h_y[i] = 2.0f;
    }

    float *d_x, *d_y;
    cudaMalloc(&d_x, size);
    cudaMalloc(&d_y, size);

    cudaMemcpy(d_x, h_x, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_y, h_y, size, cudaMemcpyHostToDevice);

    int threadsPerBlock = 256;
    int blocksPerGrid = (n + threadsPerBlock - 1) / threadsPerBlock;

    saxpy<<<blocksPerGrid, threadsPerBlock>>>(n, a, d_x, d_y);
```

```

cudaMemcpy(h_y, d_y, size, cudaMemcpyDeviceToHost);

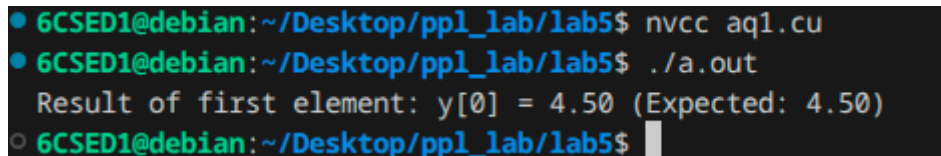
printf("Result of first element: y[0] = %.2f (Expected: 4.50)\n", h_y[0]);

cudaFree(d_x);
cudaFree(d_y);
free(h_x);
free(h_y);

return 0;
}

```

OUTPUT]



```

6CSED1@debian:~/Desktop/pp1_lab/lab5$ nvcc aq1.cu
6CSED1@debian:~/Desktop/pp1_lab/lab5$ ./a.out
Result of first element: y[0] = 4.50 (Expected: 4.50)
6CSED1@debian:~/Desktop/pp1_lab/lab5$

```

Q2]

2. Write a program in CUDA to sort every row of a matrix using selection sort.

CODE]

```

#include <stdio.h>
#include <cuda_runtime.h>

__global__ void selectionSortRows(int *matrix, int rows, int cols) {
    int rowIdx = blockIdx.x * blockDim.x + threadIdx.x;

    if (rowIdx < rows) {
        int *row = &matrix[rowIdx * cols];

        for (int i = 0; i < cols - 1; i++) {
            int minIdx = i;
            for (int j = i + 1; j < cols; j++) {
                if (row[j] < row[minIdx]) {
                    minIdx = j;
                }
            }
            int temp = row[minIdx];

```

```

        row[minIdx] = row[i];
        row[i] = temp;
    }
}

int main() {
    int rows = 4, cols = 5;
    size_t size = rows * cols * sizeof(int);

    int h_matrix[] = {
        5, 3, 8, 1, 2,
        9, 0, 4, 7, 6,
        2, 8, 1, 5, 3,
        6, 1, 9, 2, 4
    };

    printf("Initial unsorted matrix:\n");
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            printf("%d ", h_matrix[i * cols + j]);
        }
        printf("\n");
    }

    int *d_matrix;
    cudaMalloc((void**)&d_matrix, size);
    cudaMemcpy(d_matrix, h_matrix, size, cudaMemcpyHostToDevice);

    int threadsPerBlock = 256;
    int blocksPerGrid = (rows + threadsPerBlock - 1) / threadsPerBlock;

    selectionSortRows<<<blocksPerGrid, threadsPerBlock>>>(d_matrix, rows, cols);

    cudaMemcpy(h_matrix, d_matrix, size, cudaMemcpyDeviceToHost);

    printf("Sorted Matrix:\n");
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            printf("%d ", h_matrix[i * cols + j]);
        }
        printf("\n");
    }

    cudaFree(d_matrix);
    return 0;
}

```


OUTPUT]

```
6CSED1@debian:~/Desktop/pp1_lab/lab5$ nvcc aq2.cu
6CSED1@debian:~/Desktop/pp1_lab/lab5$ ./a.out
Initial unsorted matrix:
5 3 8 1 2
9 0 4 7 6
2 8 1 5 3
6 1 9 2 4
Sorted Matrix:
1 2 3 5 8
0 4 6 7 9
1 2 3 5 8
1 2 4 6 9
6CSED1@debian:~/Desktop/pp1_lab/lab5$
```